AgentProf: Semantic Profiling for AI Agents

Abstract

AI agents now orchestrate complex, multi-step activities across hundreds of sessions in production deployments, and developers need aggregate views to answer operational questions: where does the token budget go by task category, where do failures concentrate, and which workflows waste effort. Existing observability tools provide per-session debugging views but no cross-session profiling layer. The profiling methodology that answers these questions for traditional software—fold identical call stacks into flame graphs—does not directly apply because agent activities lack deterministic identifiers and runtime call-stack hierarchies. We propose a semantic profiling model that adapts this methodology: all activities are represented as uniform *semantic operations*, *intent attribution* derives deterministic labels from natural-language fields, and *profile projections* construct aggregation hierarchies from field selections at query time. AgentProf implements this model as a profiler that compiles local agent traces into pprof-compatible profiles with pluggable intent-attribution backends. On 325 real agent sessions, semantic profiling separates over 90% of system-effect weight that session-level views merge. On four public datasets with hidden ground-truth labels (34,539 operations), profiler-ranked groups localize labeled problems at 9.4% inspection cost versus flat summaries, with 45% fewer groups than fixed-session drilldown.

1 Introduction

AI agents orchestrate heterogeneous activities, including prompts, LLM calls, tool invocations, GUI actions, and system effects. As deployments scale to hundreds of sessions, developers need cross-session aggregate views to identify which task categories consume the most budget, where failures concentrate, and which workflows waste effort.

However, no existing tool provides cross-session resource attribution by semantic category. In a real development project, `cargo test` appears 2,903 times across 325 sessions, yet no observability tool can attribute these executions to the review, debugging, or refactoring prompts that triggered them. The same gap affects failure and safety analysis because unsafe operations may cluster in specific task phases, but no tool surfaces this without manual inspection of individual sessions.

The aggregation problem itself is not new. CPU profilers solved it for programs decades ago. Flame graphs [11] fold millions of function-call samples into a single chart where width represents cost. The key enabler is *deterministic* function names that merge identical call stacks. Agent traces break this precondition because prompts are natural language, and two prompts expressing the same intent share no characters (§2).

Existing agent observability tools do not solve the aggregation problem. LangSmith [13], Langfuse [14], Phoenix [1], and OpenTelemetry GenAI [24] organize events into per-execution span trees, enabling single-run debugging but not cross-session aggregation. Datadog LLM Observability [6] clusters user messages by topic, and Laminar [12] extracts structured signals from traces. Both characterize *input distributions* ("30% of users ask code questions") but not *resource attribution* ("code-review tasks consume 40% of the token budget"), because clustering at the request level does not propagate labels to downstream system effects.

We observe that three interdependent elements restore cross-session aggregation for agent traces. A uniform *operation* record that spans both agent intent and system effects replaces the missing call-stack sample. A query-time *operation stack* replaces the call stack with a field projection chosen at query time. And *intent recognition* restores the deterministic labels that flame-graph folding requires.

This paper presents AgentProf, a Rust profiler built on this model. On 325 real Codex/Claude sessions, semantic labels separate over 90% of system-effect weight that flat views merge. On four public datasets with hidden ground-truth labels, operation-stack profiling reduces inspection work to 9.4% versus flat summaries. Group fragmentation drops by 45% versus fixed-session drilldown.

This paper makes three contributions:

(1) **A profiling model** that defines a uniform operation record spanning agent intent and system effects, a query-time operation stack, and an intent-recognition step for restoring deterministic labels (§2–§3).
(2) **System: AgentProf.** A profiler that realizes this model with pluggable intent recognition (regex rules, local LLM tagging, unsupervised clustering) and produces output (pprof, SVG, JSON) consumable by both developers and automated analysis agents (§4).
(3) **Evaluation via hidden-label localization.** We score profiler output against hidden ground-truth trajectory labels (failure, safety, redundancy, and human-task boundaries), analogous to injecting known bottlenecks in CPU profiler evaluation (§5).

2 Background and Motivation

2.1 AI Agent Observability

AI agents orchestrate heterogeneous activities that span two layers: (1) high-level intent (prompts, LLM calls, tool invocations) and (2) low-level system effects (process spawns, file and network I/O). Existing observability tools cover these layers separately. LangSmith [13], Langfuse [14], Phoenix [1], and OpenTelemetry GenAI [24] organize agent-level events into per-execution *span trees*. System monitors show process and file activity without semantic attribution. The span-tree design is effective for debugging. A developer can locate a failing span, inspect its attributes, and trace its parent chain. AgentSight [34] bridges the two layers by correlating eBPF-captured system events with intercepted LLM traffic, but the resulting traces still lack aggregate profiling.

2.2 Profiling vs. Debugging

Debugging inspects a single execution, but profiling aggregates across executions to answer questions like "where does the budget go?" CPU profilers solved this aggregation problem decades ago. A flame graph [11] represents a profile as weighted *samples*, each attached to a *call stack*. Identical stacks merge, and width represents aggregate cost. The key enabler is *deterministic* function names, so the same code path always produces the same frames.

Agent activities break this precondition in two ways. First, prompts are natural language and lack deterministic identifiers. Second, there is no inherent call stack. Agent events have no runtime-determined parent-child nesting analogous to function calls. Pprof's `tagroot`/`tagleaf` options add label-derived pseudo-frames to an existing execution stack, but agent activities have no execution stack to augment.

2.3 The Aggregation Gap

Existing agent observability tools are designed for debugging, not profiling, and this creates three concrete gaps.

First, sessions are boundaries, not categories. A single session may contain review, debugging, and refactoring phases, and splitting by session assigns all of them to the same bucket. Two sessions performing the same task produce separate buckets that cannot be merged without external logic.

Second, span names describe mechanisms (`LLMCall`, `ToolInvoke`, `ChatCompletion`), not user intent (`review`, `debug`, `test`). Grouping by span name tells you how many LLM calls occurred, not which categories of work consumed the budget.

Third, these limitations compound in practice. A common system effect like running a test suite may appear thousands of times across sessions, but a flat summary shows only a single counter with no way to attribute executions to the semantic intent that triggered them. We quantify this in RQ1 (§5).

These observations yield three design requirements:

- **R1: Uniform record type.** Agent activities mix prompts, LLM calls, tool invocations, GUI actions, process events, and benchmark labels. The profiler needs a single record type that can represent all of them without type-specific objects.
- **R2: Query-time aggregation.** The same data must support multiple aggregation shapes (by task, phase, action, session, or dataset) without rebuilding the input. Sessions and spans must be fields, not fixed hierarchy nodes.
- **R3: Deterministic labels for folding.** Free-form prompts must be mapped to short, stable, repeatable labels before folding. This *intent recognition* step has no analogue in CPU profiling, where function names are already deterministic.

3 Design

The three requirements (a uniform record type, query-time aggregation, and deterministic labels for folding) drive the design. The profiling pipeline has five stages: (1) parse raw traces into operations, (2) label operations via intent recognition or mapping rules, (3) filter by a user-specified predicate, (4) project each operation onto a stack frame sequence chosen at query time, and (5) fold operations with identical stacks, summing their weights. We introduce two abstractions that satisfy R1 and R2, operations (§3.1) and operation stacks (§3.2), and a pluggable intent-recognition mechanism (§3.3) that satisfies R3.

Table 1: Built-in views. Each shares the same operation set but selects different activities, stack structures, and weights.

View	φ (select)	w (width)	Question
`tokens`	LLM calls	token count	Budget distribution?
`time`	All timed ops	duration (s)	Time distribution?
`files`	File-effect ops	event count	Repo coverage?
`network`	Net-effect ops	event count	External contacts?

3.1 Operations (R1: Uniform Record Type)

Because agent activities span both the intent and system-effect layers described in §2, a profiler should not distinguish between them at the object-model level. AgentProf therefore represents every activity as a single abstraction, the `operation`, a weighted record with string fields and additive measures.

Each operation carries string fields (`project`, `agent`, `session`, `prompt_tag`, `kind`, `model`, `path`, `domain`, `status`, ...) and additive measures (token count, duration, occurrence count). A prompt, an LLM call, a tool invocation, a file read, a GUI click, and a process event are all operation records that differ only in which fields are populated.

3.2 Operation Stacks (R2: Query-Time Aggregation)

An `operation stack` is an ordered list of fields chosen at query time. Given a stack specification $[f_1, f_2, \ldots, f_k]$, each operation o is projected to a frame sequence $\langle o.f_1, o.f_2, \ldots, o.f_k \rangle$. Operations with identical frame sequences are merged and their weights are summed. The result is the standard folded-stack representation:

```
CPU:   main;parse;tokenize         42
Agent: prompt:debug;call:llm;tool:bash  4200
```

Unlike CPU profiling, where the call stack is determined by execution, the stack is determined by the query. The same operations can be folded as a flat task summary, a fixed-session tree, a semantic phase/action hierarchy, or a dataset-native trajectory. Changing the fields changes the aggregation without changing the data. Failure, safety, and quality labels are ordinary fields that participate in the same projection, so the profiler handles diagnostic views without a separate mechanism.

Query-time stack selection directly addresses the fixed-hierarchy problem from §2. Sessions and spans are optional operation fields, not structural skeleton nodes. Including `session` in the stack produces a per-session drilldown (equivalent to a trace tree). Excluding it produces an aggregate view that existing trace tools do not natively support. The same data supports both debugging and profiling views.

A *view* is a triple (φ, σ, w): a predicate φ selects which operations participate, a stack function $\sigma = [f_1, \ldots, f_k]$ maps each operation to its frame sequence, and a weight function w assigns a nonnegative measure. The view formalism supports arbitrary user-defined views. Four built-in defaults (Table 1) answer common questions.

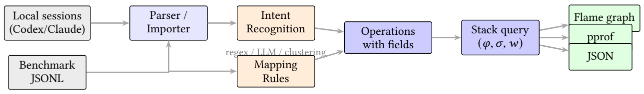

Figure 1: System architecture. Local agent histories enter through `agent-session` parsing. Public datasets enter as operation JSONL. Both paths produce operations. The stack query and output are identical.

3.3 Intent Recognition (R3: Deterministic Labels)

Operations and stacks handle heterogeneity and flexible aggregation, but folding still requires deterministic frame labels, the precondition that agent prompts violate (§2). AgentProf solves this by mapping free-form prompts to short, stable labels before stack construction.

The design uses a pluggable tagger framework with three backends. First, *regex rules* are the production default. Rules have the form `KIND:TAG=REGEX`, where `KIND` is `prompt`, `llm`, or `all`, `TAG` is a short lowercase label, and `REGEX` is a pattern. Rules are evaluated in order, and the first match wins. Humans and agents collaborate on the workflow. The developer runs AgentProf, observes the unmatched rate and sample prompts, adds rules, re-runs, and iterates until unmatched drops below 5%. This typically takes 5–10 rounds. The resulting rule set is deterministic, reproducible, and version-controllable. Second, a *local LLM tagger* handles exploration or complex multilingual prompts. A local model (e.g., a 3B-parameter GGUF via llama.cpp) generates a single-word label per prompt. Grammar-constrained decoding ensures output validity. Labels are cached, so re-runs with unchanged prompts incur zero LLM calls. Third, *unsupervised clustering* via TF-IDF vectorization and K-Means discovers natural prompt groups without predefined rules, feeding rule authoring.

The one-word label design is deliberate. Flame-graph frames must be short because long summaries make charts unreadable. Labels serve navigation and aggregation, not correctness verdicts.

For structured traces such as public benchmark datasets, where action types, tool names, and quality annotations are explicit fields, intent recognition is unnecessary. AgentProf supports deterministic *mapping rules* that derive new operation fields from existing ones before stack construction. For example, the rule `phase:navigate=type:(click|scroll|key)` maps raw action types to a coarser `phase` field, enabling cross-dataset aggregation without modifying the input.

The intent-recognition and mapping-rule paths are both pluggable. New backends (e.g., embedding-based classifiers, external LLM APIs, or domain-specific heuristics) can be added without changing the operation or stack abstractions. A *profile spec* records the complete analysis configuration (data sources, backend choice, mappings, predicates, stack fields, and outputs) for deterministic replay. Both paths produce operations with fields, and the stack query works identically on both.

4 Implementation

AgentProf is an offline, post-hoc profiler implemented as a Rust CLI (Figure 1). It reads recorded traces, not live events. The `agent-session` crate reads Codex and Claude Code local JSONL history files and reconstructs prompts, LLM calls, tool invocations, and their triggered file and network effects. Each prompt opens a span. LLM calls, tool calls, and system effects within that span inherit its semantic label, connecting intent to system behavior without runtime instrumentation. Public benchmark datasets enter as operation JSONL with mapping rules deriving fields before stack construction. Standard Chrome Trace Event format can also be imported, with non-AgentProf protocol fields preserved as operation fields.

Output formats include pprof protobuf (compatible with `go tool pprof -http`), folded-stack text (compatible with `flamegraph.pl`), standalone SVG flame graphs, and JSON summaries. All outputs are deterministic given the same input and profile spec.

AgentProf is the offline profiling component of the AgentSight [34] observability framework. AgentSight provides real-time eBPF-based capture of SSL traffic, process events, and file/network effects. AgentProf aggregates the recorded traces into semantic profiles. The two are complementary. AgentSight answers "what happened during this run" (debugging), AgentProf answers "where did the budget go across runs" (profiling).

5 Evaluation

We use two classes of data. *Real agent sessions*: 325 readable local sessions (198 Codex, 50 Claude, 77 Claude subagent) from a multi-month development project, containing 183,714 system observations. *Public labeled traces*: 15 families covering web, mobile, desktop, software, API, and dialogue agents (Table 2), totaling 47,590 operations through mapping rules. All public traces are real agent or human executions collected and labeled by their respective benchmark projects. Four oracle-rich families (AgentRewardBench, SATraj-OS, AgentNet, OSWorld-Human) provide hidden ground-truth labels for RQ2. We measure semantic separation (mixed-weight percentage), localization quality (AP, recall, work-to-first-positive), and label agreement (V-measure, boundary F1).

5.1 RQ1: Does Semantic Profiling Separate What Flat Views Mix?

The strongest argument against semantic profiling (aggregation by intent-derived labels) is that traditional tools "can already see the same information." RQ1 tests this claim directly by measuring how much system-effect weight falls into *mixed-intent* buckets (those containing observations from multiple semantic regions) under different projection choices.

Table 3 shows the result on 325 real Codex/Claude sessions. Without semantic labels, 90.4% of system-effect weight falls into mixed buckets. A flat `cargo test` counter shows 2,903 executions but cannot distinguish whether they came from `review`, `debug`, `refactor`, or `test` prompts. Adding only the prompt label reduces mixed weight to 36.7% and residual to 7.5%. The session label alone barely helps (84.4%), because sessions typically contain multiple intent phases.

A label-permutation test (1,000 shuffles, $p = 0.001$) confirms the separation is not random. Label quality is validated in RQ3.

RQ1 also tests whether query-time stack selection (R2) adds value beyond flat label grouping. The same two-abstraction model covers all 15 public trace families (Table 2) without type-specific

Table 2: Public labeled trace families. All traces are real agent or human executions. The bottom four rows provide oracle labels for RQ2.

Family	Native labels	Access	Trace origin	Eval. role
WebLINX [20], Mind2Web [7], Agent-Trek [30], WebShop [32]	Web actions, tasks, rewards	HF / public	Real web agents	Coverage
API-Bank [15], Tool-Bench [25], tau-bench [33]	API calls, outcomes	HF / JSONL	Real API agents	Coverage
SWE-agent [31]	Issue trajectories	HF viewer	Real coding agent	Coverage
AndroidCtrl [16], GUI-Odyssey [19], Scale-CUA [18]	Mobile/GUI actions	Public/HF	Real mobile agents	Coverage
AgentRewardBench [21]	Success, side-effect, looping	HF annotations	Real BrowserGym agents	RQ2 tasks
SATraj-OS [5]	Safety, reward, attack type	HF config	Real desktop agents	RQ2 task
OSWorld-Human [29]	Human grouped-action traces	GitHub JSON	Real human OS use	RQ2 task
AgentNet [27]	Step correctness, redundancy	HF JSONL	Real desktop agents	RQ2 tasks

Table 3: Semantic-axis ablation on 183,714 system observations from 325 real sessions. All rows preserve the same total weight.

Projection	Unique stacks	Mixed wt.%	Residual%
No semantic labels	11,967	90.4	44.7
Session label only	15,027	84.4	33.4
Prompt label only	24,703	36.7	7.5
Session + prompt	26,829	0.0	0.0

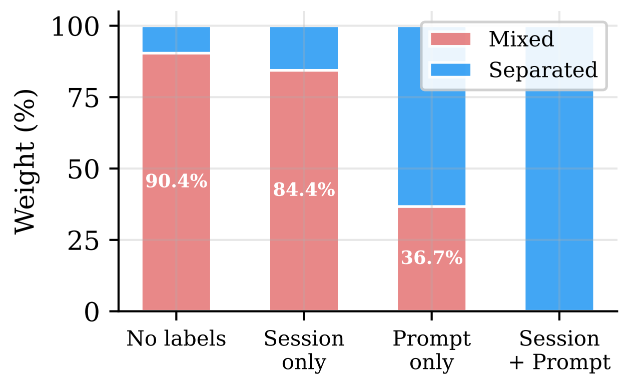

Figure 2: RQ1: Semantic-axis ablation on 183,714 system observations. Adding prompt labels reduces mixed weight from 90.4% to 36.7%. Session labels alone barely help (84.4%).

objects, validating R1. On the four oracle-rich families, the same 13,265 operations fold from 9 dataset stacks to 57 phase stacks, 226 semantic stacks, 455 action stacks, or 3,757 fixed-session stacks by changing only the stack fields. A flat GROUP BY on a single label cannot produce these views because different stack depths answer different questions (task-level budget vs. action-level duplication vs. session-level drilldown), and the profiler produces all of them from the same operations.

The multi-view design is further validated by metric divergence. The `time` and `tokens` views of the same sessions share only 7/10 top prompt categories (Spearman $\rho = 0.623$). `network`-tagged prompts rank 8th by time (I/O wait) but 93rd by tokens.

Figure 2 visualizes the ablation. Figures 3 and 4 show the resulting flame graphs for the `tokens` and `files` views respectively, showing that changing only the stack fields and weight function produces different aggregates from the same operations.

5.2 RQ2: Can Profiler Groups Localize Hidden Labeled Problems?

RQ2 asks whether the profiler's top groups correspond to real labeled problems, specifically whether categories that concentrate failures, safety violations, or wasted effort surface at the top of ranked output. The methodology is analogous to injecting known bottlenecks in a CPU profiler evaluation. Public benchmark datasets provide ground-truth labels (failure, safety violation, redundant step, human-task boundary). We hide these labels, run the profiler, rank the resulting groups, and check whether the top groups contain the hidden positives.

The benchmark uses six tasks across four datasets (AgentRewardBench looping and side-effect, SATraj-OS unsafe operation, AgentNet incorrect and redundant step, and OSWorld-Human group start), totaling 34,539 operations and 3,699 positives. We compare six grouping strategies (flat summary, fixed-session drilldown, dataset-native hierarchy, raw-action grouping, operation-stack profiling, and an oracle that uses the hidden labels) and rank groups by the fraction of hidden positives they contain.

Table 4 summarizes the results. Flat summaries recover all positives but require inspecting everything (Work@5 = 1.0). Fixed-session drilldown finds a first positive quickly (WTFP = 0.004) but fragments the workload into 285 groups with only 2.3% top-five recall. Operation-stack profiling inspects 9.4% of the work in the top five groups, recovers 39.0% of positives within 30% of the inspection budget, and reduces groups to 157.5.

Figure 5 visualizes this tradeoff: operation-stack profiling sits in the low-work, moderate-recall region. The comparison also serves

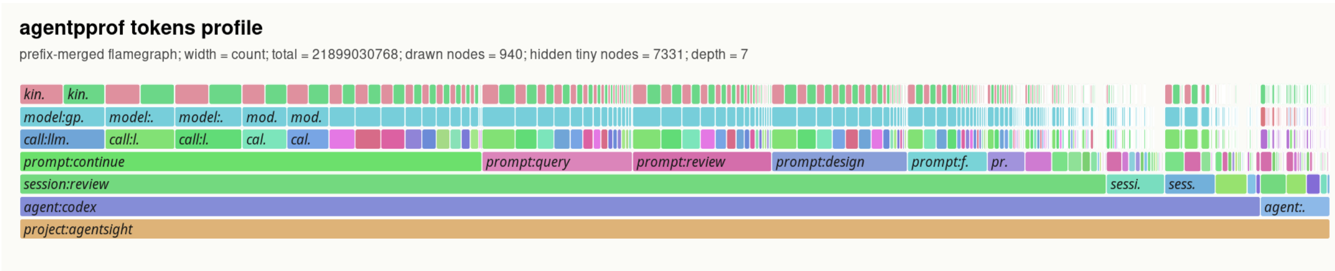

Figure 3: Flame graph of the tokens view across 325 real sessions. Width represents token count. The bottom frames show project and agent; middle frames show sessions and prompt labels (continue, query, review, design); top frames show individual LLM calls and models. The same data produces different views by changing the stack fields (Table 1).

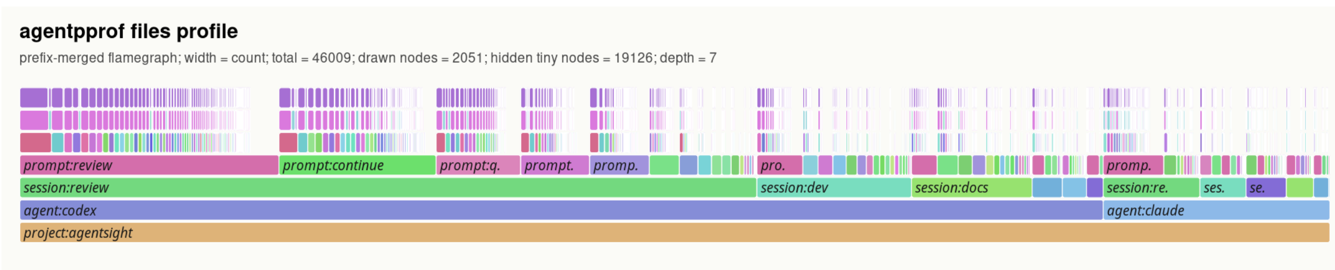

Figure 4: Flame graph of the files view across the same sessions. Width represents file-operation count. prompt:review and prompt:continue dominate file activity. Changing only the stack fields and weight function produces a different aggregate without re-parsing.

Table 4: RQ2 hidden-label localization (medians across six tasks). AP = average precision. R@5 = recall in top 5 groups. Work@5 = fraction of total work in top 5 groups. R@30% = recall at 30% inspection budget. WTFP = work-to-first-positive (fraction inspected before finding one positive). Hidden = 1 means the policy uses hidden labels (oracle upper bound). "—" = group count varies by task.

Policy	Hidden	AP	R@5	F1@5	Work@5	R@30%	WTFP	Groups
flat:width	0	0.168	1.000	0.287	1.000	0.000	1.000	1.0
fixed-session:query-aware	0	0.348	0.023	0.043	0.016	0.356	0.004	285.0
dataset-native:query-aware	0	0.357	0.867	0.282	0.854	0.338	0.058	—
raw-action:query-aware	0	0.274	0.244	0.220	0.151	0.333	0.037	—
op-stack:query-aware	0	0.312	0.188	0.198	0.094	0.390	0.038	157.5
op-stack:oracle-upper	1	0.599	0.300	0.403	0.044	0.564	0.012	—
label-drilldown:oracle	1	1.000	0.670	0.797	0.115	1.000	0.038	—

as evidence for the two-abstraction model (§3). Fixed-session drill-down and dataset-native hierarchy are alternative abstractions that fix the hierarchy at session or dataset boundaries. Operation stacks subsume both as special cases (include `session` in the stack to recover fixed-session; include `dataset` to recover dataset-native). On localization, operation stacks improve top-five recall over fixed-session on 5/6 tasks, reduce median group count by 45%, and save 90% of inspection work versus flat summaries. Fixed-session drill-down retains lower work-to-first-positive on 4/6 tasks, confirming that profilers and debuggers are complementary.

Statistical validation supports these comparisons. Bootstrap (10,000 resamples) and label-permutation (2,000 trials) confirm that the AP differences are stable and exceed the 95% confidence threshold on all six tasks.

Beyond localization, we test whether profiler output suggests concrete configuration improvements. We modify stack fields, mapping rules, or rankers in the profile spec and re-run on the same operations. Five of six changes improve AP, top-five recall, and work-to-first-positive (median AP improvement 0.038). Three reusable patterns emerge: (1) safety/side-effect tasks benefit from field and ranker changes; (2) step-quality tasks benefit from stack-based localization followed by fixed-session drilldown for examples; (3) human-boundary tasks require boundary-derived fields. The OSWorld-Human task shows that features visible without the hidden labels are insufficient for detecting task boundaries. Adding fields derived

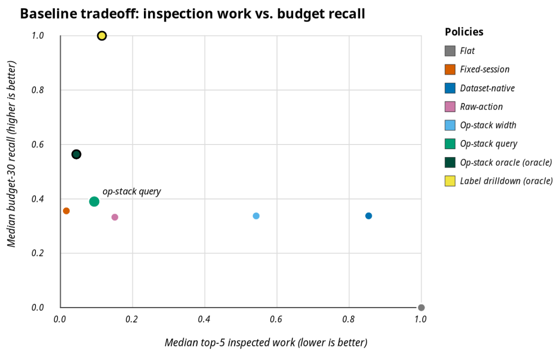

Figure 5: RQ2: Inspection work vs. recall at 30% inspection budget. Operation-stack profiling (labeled) achieves low work (0.094) with moderate recall (0.39), while flat requires full inspection and fixed-session fragments into 285 groups. Oracle policies (large dots) are upper bounds.

from the hidden boundary annotations improves AP from 0.240 to 0.258 and reduces groups from 108 to 74, confirming that boundary detection requires oracle information.

Ablating rank features reveals that 7 features (including `repeat_signal` and write-action) drive localization, while 3 mislead the ranker. Manually correcting the misleading features improves AP on 2/3 cases, but the correction requires offline scoring rather than automatic learning.

5.3 RQ3: How Reliable Are the Derived Labels?

RQ1 and RQ2 assume that semantic labels are useful. RQ3 tests the label-derivation pipeline itself. For structured traces (public datasets), we evaluate the *mapping* path against ground-truth annotations. For free-form traces (real sessions), we evaluate the *intent-recognition* path at the scale and syntax level. Quality assessment of the LLM/regex tagger against free-form ground truth remains future work.

On 325 real sessions, the LLM tagger processes 118,021 tag requests with zero failures and p95 latency of 31 ms per label.

Public benchmark datasets provide native action labels that serve as ground truth for mapping quality. We run a leave-dataset-out evaluation. For each of 9 datasets, we train mapping rules on the other 8 and measure V-measure between the mapping-derived `phase` field and the dataset's native `action` labels, plus boundary F1 between mapped phase boundaries and native action boundaries.

Table 5 and Figure 6 show the results. On 7/9 datasets, V-measure exceeds 0.7, indicating strong agreement between mapping-derived and native labels. The two low-scoring datasets (toolbench, api-bank) have atypical action vocabularies that the held-out mapping rules do not cover.

A supervised boundary predictor further validates the mapping approach. On OSWorld-Human held-out adjacent pairs, a learned boundary predictor achieves F1 0.77, beating the best simple baseline (`always_boundary`, F1 0.71). Learned predictors beat simple baselines on 4/5 tested datasets, while AgentRewardBench looping is fully explained by a single repeat-signal feature (baseline F1 1.0). These comparisons confirm that mapping rules produce useful labels but are not automatic intent discovery.

All results are backed by profile specs for deterministic replay. We re-execute 76 tracked specs twice (152 invocations), and all produce identical output with median runtime 1.6 s per spec.

Table 5: Leave-dataset-out mapping quality. V-measure between mapping-derived `phase` and native `action` labels. 13,265 operations from the four oracle-rich families across 9 held-out evaluations.

Held-out dataset	Ops	V-measure	Boundary F1
mind2web	774	1.000	1.000
webshop-expert	2,504	1.000	1.000
swe-agent	496	0.926	0.962
weblinx-chat	500	0.872	0.860
agenttrek	200	0.862	—
gui-odyssey	7,868	0.811	0.842
android-control	9	0.716	0.727
toolbench	866	0.134	0.353
api-bank	48	0.000	—

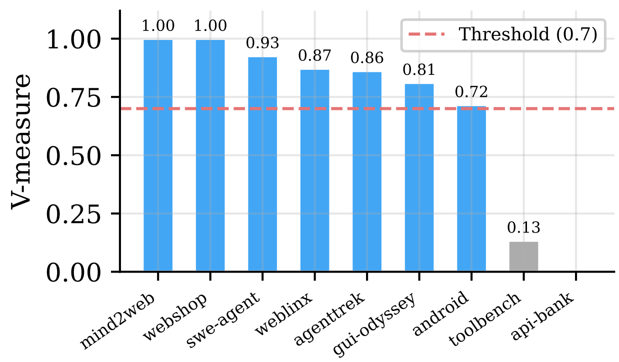

Figure 6: RQ3: Leave-dataset-out V-measure between mapping-derived `phase` and native `action` labels. The dashed line marks 0.7. Seven of nine datasets exceed this threshold.

6 Related Work

Flame graphs [11] and pprof [10] establish the folded-stack lineage. Profiles are weighted samples with location hierarchies. Pprof supports sample labels and can visualize them as pseudo stack frames through tag-root/tag-leaf options. Perfetto generalizes trace ingestion with SQL-based analysis and derived events [9]. These systems support query-time aggregation, so AgentProf does not claim novelty for flexible aggregation alone. The difference is structural. Pprof's tagroot adds label-derived frames to an existing execution call stack. Agent traces have no execution stack, so the entire profiling structure must be constructed from semantic fields. The evaluation also differs: AgentProf scores profiler output against hidden agent-trajectory labels, a methodology with no analogue in CPU profiling.

OpenTelemetry GenAI and OpenInference standardize GenAI spans [2, 24]. LangSmith [13], Langfuse [14], and Phoenix [1] provide trace visualization, session management, evaluations, and dashboards. AgentOps [8] argues for structured agent observability covering goals, plans, tools, and artifacts. AgentTelemetry [3] defines 14 fault types over 9 agent-specific span kinds that extend OpenTelemetry for agent orchestration phases. Wang et al. [28] survey evidence tracing and execution provenance as foundations for agent accountability. These systems excel at single-execution debugging or fault detection. AgentProf complements them with cross-session aggregate profiling. Datadog LLM Observability [6] and Laminar Signals [12] take steps toward semantic aggregation by clustering user messages or extracting structured events, but they characterize *input distributions*, not *resource attribution*.

AgentRx [4] releases annotated failure trajectories and uses constraint checking plus an LLM judge for root-cause localization. TEL-Bench [26] builds span-localization benchmarks from semantic error annotations. AgentAtlas [22] argues that outcome leaderboards miss trajectory-level quality. TrajAD [17] and AgentFixer [23] detect trajectory anomalies and recommend fixes. These systems perform *span-level* or *step-level* fault localization within a single execution. AgentProf performs *category-level* aggregate profiling, identifying which categories of work produce the most failures, consume the most budget, or generate repeated system effects across sessions.

7 Conclusion

Agent observability needs profiling, not only debugging. AgentProf shows that two abstractions, operations and operation stacks, close the aggregation problem once intent recognition restores deterministic labels. Scoring profiler output against hidden trajectory labels confirms that semantic profiling separates over 90% of mixed system-effect weight, localizes labeled problems at 9.4% inspection cost versus flat summaries, and produces labels that agree with ground truth on 7/9 held-out datasets. Fixed-session drilldown remains faster to a first positive on 4/6 tasks, confirming that profilers and debuggers are complementary, and AgentProf supports both by including or excluding session fields in the stack query. The main limitations are single-project real-session data and placeholder intent-recognition quality on free-form prompts. Multi-project evaluation and trace-ecosystem integration are the most impactful next steps.

References

[1] Arize AI. 2024. Arize Phoenix. https://arize.com/docs/phoenix.
[2] Arize AI. 2024. OpenInference Specification. https://arize-ai.github.io/openinference/spec/.
[3] Sai Sree Harsha Balusu. 2026. AgentTelemetry: A Fault Detection Benchmark and Toolkit for LLM Agent Observability. In *Proceedings of the 3rd ACM International Conference on AI-Powered Software (AIware '26)*. https://dl.acm.org/doi/10.1145/3805760.3814931
[4] Shraddha Barke, Arnav Goyal, Alind Khare, Avaljot Singh, Suman Nath, and Chetan Bansal. 2026. AgentRx: Diagnosing AI Agent Failures from Execution Trajectories. arXiv preprint arXiv:2602.02475. doi:10.48550/arXiv.2602.02475
[5] Xinquan Chen et al. 2026. Safactory: A Scalable Agentic Infrastructure for Training Trustworthy Autonomous Intelligence. arXiv preprint arXiv:2605.06230. doi:10.48550/arXiv.2605.06230
[6] Datadog. 2025. LLM Observability. https://docs.datadoghq.com/llm_observability/.
[7] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2Web: Towards a Generalist Agent for the Web. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*.
[8] Liming Dong, Qinghua Lu, and Liming Zhu. 2024. AgentOps: Enabling Observability of LLM Agents. arXiv preprint arXiv:2411.05285. doi:10.48550/arXiv.2411.05285
[9] Google. 2024. Perfetto Trace Processor. https://perfetto.dev/docs/analysis/trace-processor.
[10] Google. 2024. pprof. https://github.com/google/pprof.
[11] Brendan Gregg. 2011. Flame Graphs. https://www.brendangregg.com/flamegraphs.html.
[12] Laminar. 2025. Signals: Structured Event Extraction from Traces. https://docs.lmnr.ai/signals/introduction.
[13] LangChain. 2024. LangSmith Observability. https://docs.langchain.com/langsmith/observability.
[14] Langfuse. 2024. Langfuse Observability. https://langfuse.com/docs/observability/overview.
[15] Minghao Li, Feifan Song, Bowen Yu, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 3102–3116.
[16] Wei Li, William E. Bishop, Alice Li, Chris Rawles, Folawiyo Campbell-Ajala, Divya Tyamagundlu, and Oriana Riva. 2024. On the Effects of Data Scale on UI Control Agents. arXiv preprint arXiv:2406.03679. doi:10.48550/arXiv.2406.03679
[17] Yibing Liu, Chong Zhang, Zhongyi Han, Hansong Liu, Yong Wang, Yang Yu, Xiaoyan Wang, and Yilong Yin. 2026. TrajAD: Trajectory Anomaly Detection for Trustworthy LLM Agents. *arXiv preprint arXiv:2602.06443* (2026). doi:10.48550/arXiv.2602.06443
[18] Zhaoyang Liu et al. 2025. ScaleCUA: Scaling Open-Source Computer Use Agents with Cross-Platform Data. arXiv preprint arXiv:2509.15221. doi:10.48550/arXiv.2509.15221
[19] Quanfeng Lu, Wenqi Shao, Zitao Liu, Fanqing Meng, Boxuan Li, Botian Chen, Siyuan Huang, Kaipeng Zhang, Yu Qiao, and Ping Luo. 2025. GUI Odyssey: A Comprehensive Dataset for Cross-App GUI Navigation on Mobile Devices. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*.
[20] Xing Han Lù, Zdeněk Kasner, and Siva Reddy. 2024. WebLINX: Real-World Website Navigation with Multi-Turn Dialogue. arXiv preprint arXiv:2402.05930. doi:10.48550/arXiv.2402.05930
[21] Xing Han Lù, Amirhossein Kazemnejad, Nicholas Meade, Arkil Patel, Dongchan Shin, Alejandra Zambrano, Karolina Stańczak, Peter Shaw, Christopher J. Pal, and Siva Reddy. 2025. AgentRewardBench: Evaluating Automatic Evaluations of Web Agent Trajectories. arXiv preprint arXiv:2504.08942. doi:10.48550/arXiv.2504.08942
[22] Parsa Mazaheri and Kasra Mazaheri. 2026. AgentAtlas: Beyond Outcome Leaderboards for LLM Agents. arXiv preprint arXiv:2605.20530. doi:10.48550/arXiv.2605.20530
[23] Hadar Mulian, Sergey Zeltyn, Ido Levy, Liane Galanti, Avi Yaeli, and Segev Shlomov. 2026. AgentFixer: From Failure Detection to Fix Recommendations in LLM Agentic Systems. *arXiv preprint arXiv:2603.29848* (2026). doi:10.48550/arXiv.2603.29848
[24] OpenTelemetry. 2024. OpenTelemetry GenAI Semantic Conventions. https://github.com/open-telemetry/semantic-conventions-genai.
[25] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerber, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.
[26] Jiaming Wang, Ziteng Feng, Jiangtao Wu, Ruihao Li, Qianqian Xie, Yuxiang Ren, He Zhu, Xueming Han, Fanyu Meng, Junlan Feng, and Jiaheng Liu. 2026. Where Do Deep-Research Agents Go Wrong? Span-Level Error Localization in Agent Trajectories. arXiv preprint arXiv:2606.02060. doi:10.48550/arXiv.2606.02060
[27] Xinyuan Wang et al. 2025. AgentNet (OpenCUA Dataset). https://huggingface.co/datasets/xlangai/AgentNet.
[28] Yiqi Wang, Jiaqi Zhang, Taotao Cai, et al. 2026. From Agent Traces to Trust: A Survey of Evidence Tracing and Execution Provenance in LLM Agents. *arXiv preprint arXiv:2606.04990* (2026).
[29] WukLab. 2025. OSWorld-Human. https://github.com/WukLab/osworld-human.
[30] Yiheng Xu, Dunjie Lu, Zhennan Shen, Junli Wang, Zekun Wang, Yuchen Mao, Caiming Xiong, and Tao Yu. 2024. AgentTrek: Agent Trajectory Synthesis via Guiding Replay with Web Tutorials. arXiv preprint arXiv:2412.09605. doi:10.48550/arXiv.2412.09605
[31] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*.
[32] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. 2022. WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*.

[33] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv preprint arXiv:2406.12045. doi:10.48550/arXiv.2406.12045

[34] Yusheng Zheng, Yanpeng Hu, Tong Yu, and Andi Quinn. 2025. AgentSight: System-Level Observability for AI Agents Using eBPF. In *Proceedings of the 4th Workshop on Practical Adoption Challenges of ML for Systems (PACMI '25, co-located with SOSP)*. doi:10.1145/3766882.3767169